Lecture - AI Search on Databricks

Duration: 20 minutes

Overview

This lecture covers the theory behind AI Search before you build one in the next demo. We'll explore how embeddings and similarity search power retrieval, how AI Search is organized into endpoints and indexes and the search modes it supports, how agents use AI Search under the hood, and when to choose managed RAG vs. building your own. We'll also cover how Unity Catalog governs the entire pipeline.

Learning Objectives

By the end of this lecture, you will be able to:

1. Explain how embeddings transform text into searchable vectors
2. Describe how cosine similarity drives retrieval ranking
3. Describe the roles of AI Search endpoints and indexes
4. Compare the index types for providing embeddings (managed, self-managed, direct access)
5. Distinguish semantic, keyword, and hybrid search
6. Compare managed RAG (Knowledge Assistants) and custom RAG approaches
7. Explain how Unity Catalog governs AI Search indexes

A. Embeddings and AI Search Fundamentals

A1. What Are Embeddings?

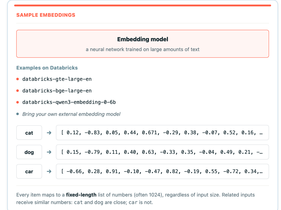
An **embedding** is a calculation (represented as a vector) that is based on the context that a word appears in. Texts with similar meanings produce vectors that are close together in mathematical space.

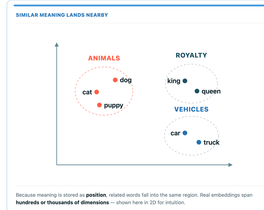
For example, "cozy apartment near the park" and "comfortable flat close to green space" would have similar embeddings, even though they use completely different words.

For example, when creating an AI Search index with `embedding_model_endpoint_name="databricks-gte-large-en"`, Databricks calls this model on every chunk in your table to produce these vectors. Now let's understand how those vectors are used to find relevant results.

Key considerations when working with embeddings:

- **Use the same model** for indexing documents and processing queries, as this ensures they exist in the same vector space. With Genie Agents, this is managed for you, but Databricks has fully support for custom workloads.
- **Respect context windows** - embedding models have token limits (e.g., 512 tokens). Text beyond the limit is silently truncated
- **Dimension trade-offs** - higher dimensions capture more nuance but cost more to store and search. The number of dimensions represents the amount of context the model considers when calculating the embeddings.





A2. The Similarity Search Pipeline

When you search a vector index, the system:

1. Converts your query text into an embedding vector
2. Finds the closest vectors in the index (using cosine similarity or other distance metrics)
3. Returns the corresponding text chunks, ranked by relevance



For fast retrieval at scale, Databricks uses **Approximate Nearest Neighbors (ANN)** algorithms, trading a small amount of precision for dramatic speed gains.

When you use `similarity_search` on an AI Search index, your query text is embedded, compared against all vectors in the index, and the top matches are returned with their similarity scores. You'll see this in action in the next demo.

> **Learn More**
> See the full AI Search documentation (AWS | Azure | GCP) for details on endpoints, indexes, and APIs.

A3. What Is AI Search?

AI Search is a retrieval solution built into the Databricks Data Intelligence Platform and integrated with Unity Catalog governance. Given a query, its core job is to find the stored embeddings that are **most semantically similar** (using vector similarity). It is the retrieval engine behind RAG systems, recommender systems, and image or video search.

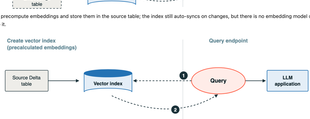
To stay fast at scale, AI Search uses the **Hierarchical Navigable Small World (HNSW)** algorithm for approximate nearest neighbor (ANN) search. This means lookups remain quick even across hundreds of millions of vectors.

Every deployment is built from two components: **endpoints** and **indexes**. We look at those next, then at how embeddings get into an index and the ways you can search it.

A4. Providing Embeddings: Index Types

How do vectors get into an index? There are three approaches. The first two keep the index automatically in sync with a Delta source; the third gives you manual control. In each diagram, the left side shows how the index is **created** and the right side shows the **query endpoint** at serving time — the numbered steps trace a single query.

Databricks-managed embeddings (Delta Sync) — you point at a text column and choose an embedding model; Databricks computes the embeddings, optionally saves them to a table, and keeps the index synced as the source changes. The simplest path, and the one used in this course. At query time, (1) the query is embedded with the same model, (2) compared against the vector index, and (3) the nearest results are returned to the app.



Self-managed embeddings (Delta Sync) — you precompute embeddings and store them in the source table; the index still auto-syncs on changes, but there is no embedding model on the create side. You **cannot later convert** a self-managed index to managed — you would recreate it.



Direct Access — you write vectors directly through the REST API or SDK, with no source table and no automatic sync. Best when an external pipeline computes embeddings or you need real-time upserts.

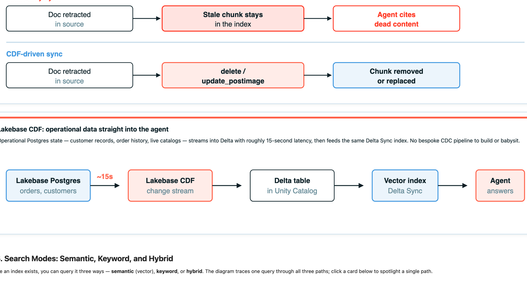


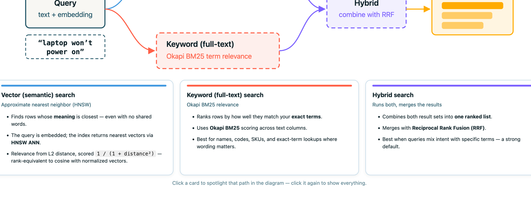
> **Full-text search index (Beta)**
> Available on **storage-optimized** endpoints only, a full-text search index needs **no embedding column**: it searches text directly with BM25 keyword matching and uses triggered sync. Useful for pure keyword retrieval when vectors are not needed.

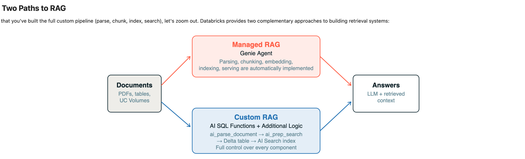
A5. CDF for RAG Workloads

A RAG agent is only as good as the index behind it. Source documents changes constantly. For example, policies are revised, tickets arrive, specs evolved, etc. When it comes to RAG workloads, there is always one crucial question that needs answering from an engineering perspective: **which chunks actually changed since the last pipeline run?** Databricks' Change Data Feed (CDF) answers this question by addressing cost and sync modes.

- **Embeddings can be expensive**: Embeddings require compute or API calls. Without change tracking, you either re-embed the whole corpus or accept a stale nightly batch. CDF turns the work into a function of change volume, not corpus size.
- **Two sync modes**: A Delta Sync index reads CDF either on demand or as a continuous stream.







A6. Search Modes: Semantic, Keyword, and Hybrid

Once an index exists, you can query it three ways — **semantic** (vector), **keyword**, or **hybrid**. The diagram traces one query through all three paths; click a card below to spotlight a single path.



Vector (semantic) search
Approximate nearest neighbor (HNSW)

- Finds rows whose **meaning** is closest — even with no shared words.
- The query is embedded; the index returns nearest vectors via **HNSW ANN**.
- Relevance from L2 distance, scored `1 / (1 + distance²)` — rank-equivalent to cosine with normalized vectors.

Keyword (full-text) search
Okapi BM25 relevance

- Ranks rows by how well they match your **exact terms**.
- Uses **Okapi BM25** scoring across text columns.
- Best for names, codes, SKUs, and exact-term lookups where wording matters.

Hybrid search
Runs both, merges the results

- Combines both result sets into **one ranked list**.
- Merges with **Reciprocal Rank Fusion (RRF)**.
- Best when queries mix intent with specific terms — a strong default.

Click a card to spotlight that path in the diagram — click it again to show everything.

B. Managed vs. Custom RAG

B1. Two Paths to RAG

Now that you've built the full custom pipeline (parse, chunk, index, search), let's zoom out. Databricks provides two complementary approaches to building retrieval systems:



Managed RAG (Genie Agents):

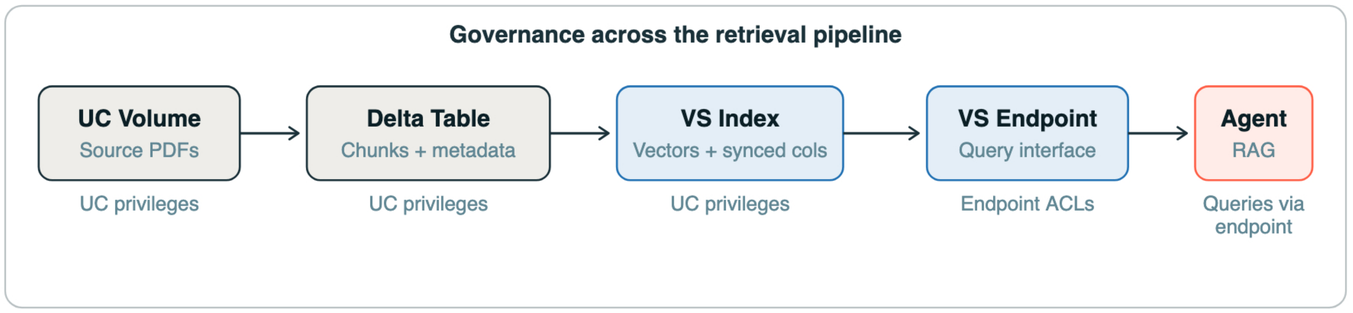
- You provide documents (in a UC Volume or AI Search Index)
- The system handles parsing, chunking, embedding, indexing, and serving
- Optimized automatically via feedback loops
- Best for: standard RAG use cases where speed to production matters

Custom RAG (Agent Framework + AI Search):

- You build the pipeline yourself with full ownership of each step
- Full control over parsing strategy, chunking approach, embedding model, and retrieval logic
- Use AI Search indexes as retrieval tools in your custom agent
- Best for: unique requirements, complex multi-step reasoning, novel architectures

B2. Governance with Unity Catalog

AI Search indexes are governed by Unity Catalog. Access to Unity Catalog objects in the retrieval pipeline is controlled with Unity Catalog privileges, while AI search endpoints are controlled with endpoint ACLs. Audit visibility is available through system tables, and lineage can be captured across Unity Catalog tables and paths used in the retrieval pipeline.



C. AI Search vs. Lakebase Search

Databricks now offers two search options for retrieval. They complement each other and serve different architectures:

	AI Search	Lakebase Search
What it is	Managed search service over Unity Catalog tables and volumes — handles embedding, indexing, and retrieval as a fully managed stack	Postgres extensions adding vector + keyword search inside a Lakebase project
Primary use case	Production search — search bars, recommendation systems, and document RAG for unstructured content (PDFs, docs, slides)	Operational search — query structured and semi-structured data already in Postgres
Used by	Search bars, recommender systems, Genie Agents, custom RAG agents, any application calling the AI Search API	Applications backed by Lakebase — customer-facing apps, internal tools, agent memory
Search modes	Vector (ANN), keyword (BM25), hybrid (RRF), and hybrid + reranker	Vector (ANN), keyword (BM25), and hybrid (RRF fusion)
JOINs with operational data	No — results come back as a separate retrieval call	Yes — search, JOIN, and filter in one SQL query, one transaction
Permissions model	Unity Catalog ACLs on the index	Postgres row-level GRANTs on the same tables
Index management	Fully managed by Databricks	You create indexes with `CREATE INDEX ... USING lakebase_ann / lakebase_bm25`
Lakehouse integration	Native — indexes sync from UC tables/volumes via Delta Sync	Via synced tables — sync a UC table into Lakebase with type mapping for search columns

When to choose which

Use **AI Search** when you want a fully managed search engine — search bars, recommendations, or document RAG — with built-in hybrid retrieval, reranking, and quality evaluation out of the box. Use **Lakebase Search** when your data already lives in Lakebase and you want low-latency search co-located with your operational tables — for example, searching customer reviews, product catalogs, or support tickets alongside structured metadata in a single SQL query.

Genie CODE

Want to learn more about AI Search and embedding strategies? Ask Genie Code. Click on the genie icon and begin querying. For example, click the **Copy** button below and paste into **Genie Code**.

```
How does Databricks AI Search handle hybrid keyword and semantic search?
```

Copy

© 2026 Databricks, Inc. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Privacy Policy | Terms of Use | Support